UNIVERSIDADE ESTADUAL DA PARAÍBA

BACHARELADO EM CIÊNCIAS DA COMPUTAÇÃO

RELATÓRIO DE TESTES: UBER - PB

CAMPINA GRANDE

2024

INTRODUÇÃO

Este documento relata de maneira detalhada os testes implementados para o sistema UberPB. O objetivo principal foi garantir a integridade das regras de negócio, a persistência correta dos dados em arquivos JSON e a confiabilidade das entidades do domínio. Os testes foram construídos com foco em isolamento, garantindo que a execução dos testes não interfira ou corrompa os dados de produção ou de uso local.

TECNOLOGIAS UTILIZADAS

Linguagem: Java JDK 22
Framework de Testes: JUnit 5 (jupiter-api 5.9.3)
Manipulação de JSON: Gson 2.13.0
Técnicas Aplicadas: Injeção de Dependências (via construtores), Java Reflection (para isolamento de repositórios) e Arquivos Temporários (File.createTempFile).

ESCOPO E COBERTURA DOS TESTES

A implementação dos testes foi dividida em 3 partes que são partes da arquitetura do sistema. Então, primeiramente foram testadas as classes de "Model" que definem os objetos a serem instanciados a fim de representar entidades do sistema. Logo em seguida, foram testadas as classes de "Repository" que são as classes responsáveis pela manipulação e armazenamento de dados de forma local em arquivos no formato JSON. Por fim, foram realizados testes para as classes responsáveis pelos serviços e funcionalidades de cada entidade, bem como as regras de negócios das entidades do sistema. Dessa forma, foram testadas as seguintes classes

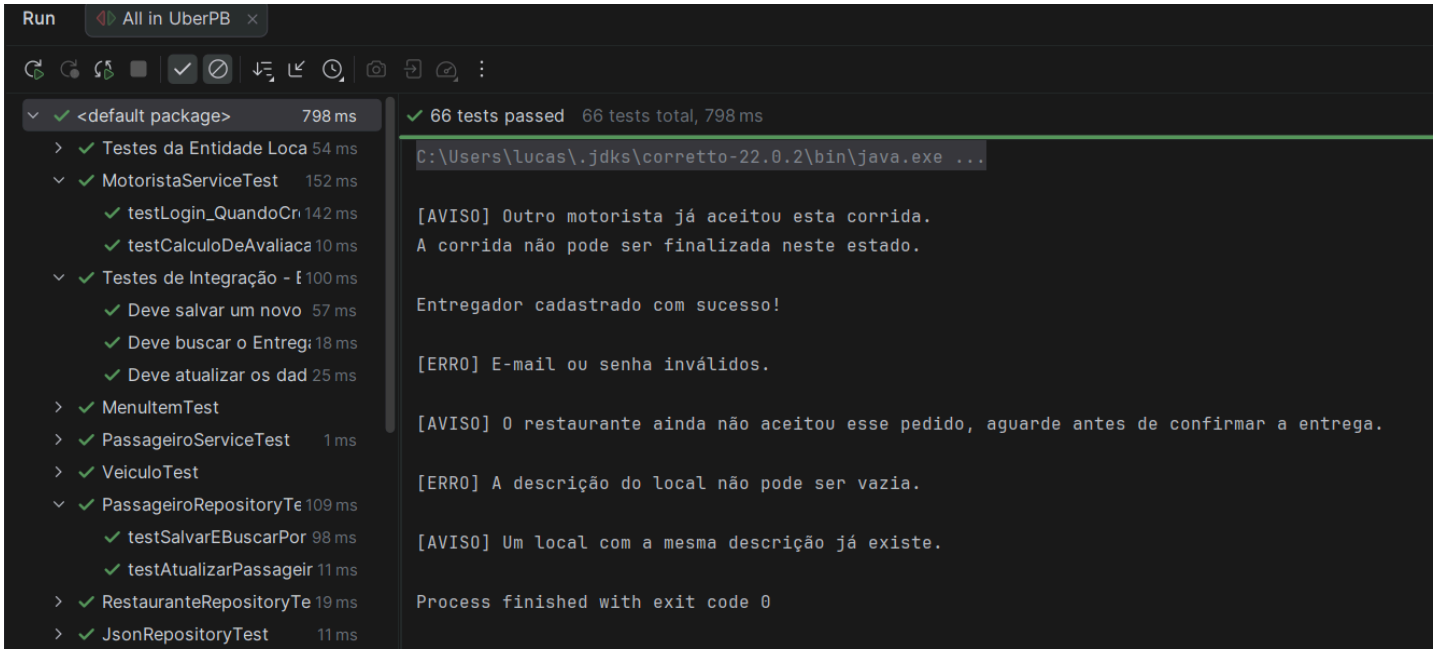
MODEL		
Entidade Testada	**Funcionalidades Testadas**	**Resultados dos Testes**
Usuario (Abstrata)	Herança, Getters e Setters comuns (Nome, Email, CPF).	Aprovado
Passageiro	Inicialização de listas de histórico e corridas/pedidos pendentes.	Aprovado
Motorista	Controle de status do motorista e vínculo de veículo.	Aprovado
Corrida	Atribuição de valores, formas de pagamento, rejeições e datas.	Aprovado

Entregador	Vínculo de motocicleta e adição de notificações de pedido.	Aprovado
Restaurante	Mapeamento de CNPJ herdando de CPF, manipulação de cardápio.	Aprovado
Pedido	Construtores dinâmicos, agendamento e transições de status do pedido.	Aprovado
Localizacao	Cálculo matemático de distância (Fórmula de Haversine), Equals.	Aprovado
Veiculo	Associação de categorias e limites de capacidade.	Aprovado

SERVICES		
Entidade Testada	**Funcionalidades Testadas**	**Resultados dos Testes**
VeiculoService	Testada a lógica de determinação de categoria com base no ano, assentos, porta-malas e status premium.	Aprovado
CorridaService	Validação do cálculo de preços estimados e Trava de segurança de status.	Aprovado
PedidoService	Cálculo correto do somatório de itens do cardápio mais taxa de entrega fixa, cálculo de tempo estimado de entrega e formatação amigável de status para o cliente final.	Aprovado
RestauranteService	Inclusão dinâmica de itens no menu com auto-incremento de ID em memória e Busca e remoção de itens específicos do cardápio.	Aprovado
MotoristaService	Cálculo progressivo da média de avaliações, tratando a primeira avaliação separadamente das subsequentes e Validação de regras de login.	Aprovado

PassageiroService	Cálculo progressivo da média de avaliações, tratando a primeira avaliação separadamente das subsequentes e Validação de regras de login.	Aprovado

REPOSITORY		
Entidade Testada	**Funcionalidades Testadas**	**Resultados dos Testes**
JsonRepository<T>	Conversão genérica de instâncias, leitura de arquivos recém-criados.	Aprovado
CorridaRepository	Injeção de caminho JSON via construtor sobrecarregado.	Aprovado
PedidoRepository	Injeção de caminho JSON via construtor sobrecarregado.	Aprovado
VeiculoRepository	Injeção de dependência via Java Reflection e auto-incremento de ID.	Aprovado
MotoristaRepository	Injeção via Reflection, atualização de dados e busca por email.	Aprovado
PassageiroRepository	Injeção via Reflection, busca por CPF e Email.	Aprovado
RestauranteRepository	Injeção via Reflection, validação e listagem geral.	Aprovado
MotocicletaRepository	Injeção via Reflection e busca por placa.	Aprovado
LocalizacaoRepository	Substituição dupla via Reflection e bloqueio de duplicatas.	Aprovado

RESULTADOS

Após implementados os testes, 66 testes foram realizados e executados, de forma que todos foram aprovados, conforme demonstrado pela seguinte imagem: